MATERIA
INF560 — Desarrollo Web Backend

GUÍA DE LABORATORIO 6

Rutas, Controladores y Vistas Blade

Material de Estudio Teórico-Práctico

CONTENIDO DE LA GUÍA

1. Rutas resource y convenciones REST en Laravel
2. Implementación de BookController con eager loading
3. Implementación de AuthorController y route model binding
4. Layout principal con Tailwind CSS y navbar
5. Vistas Blade: index, show y create de libros
6. Vistas Blade de autores con tabla y detalle
7. Componentes Blade reutilizables
8. Versionado: commit y tag v0.3.0

COMPETENCIA DE LA GUÍA

Diseñar e implementar la capa de presentación de una aplicación Laravel utilizando rutas RESTful resource, controladores con inyección de dependencias y route model binding, vistas Blade modulares con layouts, secciones y componentes anónimos, integrando Tailwind CSS para estilos y aplicando buenas prácticas de eager loading para optimizar el acceso a datos relacionados.

1. Objetivos de la guía

Al finalizar esta guía, el estudiante será capaz de:

- Configurar rutas RESTful tipo resource en Laravel y comprender la convención de nombres y verbos HTTP que se generan automáticamente.
- Crear controladores resource y diferenciar entre los siete métodos estándar: index, create, store, show, edit, update y destroy.
- Aplicar route model binding implícito para resolver modelos Eloquent directamente desde los parámetros de ruta.
- Optimizar consultas mediante eager loading con with() y load() para evitar el problema N+1 en vistas que listan recursos relacionados.
- Diseñar un layout principal Blade utilizando @yield, @section y @extends, e integrar Tailwind CSS vía CDN para construir interfaces responsivas.
- Construir vistas Blade que rendericen catálogos paginados, fichas de detalle y formularios maquetados.
- Crear componentes Blade anónimos reutilizables y entender la diferencia frente a los componentes de clase.
- Versionar el avance del proyecto con un commit semántico y un tag anotado v0.3.0.

2. Prerrequisitos

Esta guía continúa el proyecto acumulativo library-app sobre el estado alcanzado al finalizar la Guía 5. Antes de iniciar, verifica que tu proyecto cumple los siguientes puntos:

- El proyecto Laravel 13 está creado y conectado correctamente a la base de datos PostgreSQL (configurado en .env).
- Los modelos Eloquent Author, Category, Book, Member y Loan existen en app/Models/ con sus relaciones (hasMany, belongsTo, belongsToMany) ya implementadas.
- Las migraciones para users, authors, categories, books, author_book, members y loans se ejecutaron sin errores con php artisan migrate.
- Los factories y seeders generados en la Guía 5 pueblan la base de datos con datos de prueba; al ejecutar php artisan db:seed se insertan al menos 30 libros con autores, categorías y portadas asignadas.
- El repositorio Git tiene el tag v0.2.0 correspondiente a la entrega de la Guía 5 y la rama main está actualizada.
- Tienes una terminal abierta dentro del directorio del proyecto y el servidor de desarrollo se levanta sin errores con php artisan serve.

> **NOTA: Verificación rápida**
> Ejecuta los comandos a continuación. Si todo funciona, estás listo para comenzar:
> • php artisan migrate:status → muestra todas las migraciones como Ran
> • php artisan tinker → Book::with('authors')->first() → devuelve un libro con sus autores
> • git tag → debe listar al menos v0.1.0 y v0.2.0

3. Desarrollo paso a paso

3.1 Rutas resource: convención sobre configuración

En Laravel, una ruta resource es una declaración compacta que genera automáticamente las siete rutas necesarias para implementar un recurso CRUD completo siguiendo el estilo REST. En lugar de definir manualmente cada ruta para listar, crear, mostrar, editar, actualizar y eliminar, una sola línea las produce todas con verbos HTTP, URIs y nombres ya estandarizados.

> **DEFINICIÓN: REST y convención sobre configuración**
> REST (Representational State Transfer) es un estilo arquitectónico para servicios web que asocia operaciones a verbos HTTP (GET, POST, PUT/PATCH, DELETE) sobre recursos identificados por URIs.
> Convención sobre configuración: el framework asume valores por defecto razonables (nombres de rutas, controladores, parámetros) para que el desarrollador escriba menos código repetitivo y siga prácticas estándar.

Paso 1 — Editar el archivo de rutas web

Abre el archivo routes/web.php y reemplaza su contenido por el siguiente código:

```
ROUTES/WEB.PHP
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\BookController;
use App\Http\Controllers\AuthorController;

// Redirección de la raíz al catálogo de libros
Route::redirect('/', '/books');

// Rutas RESTful para libros y autores
Route::resource('books', BookController::class);
Route::resource('authors', AuthorController::class);
```

Paso 2 — Verificar las rutas generadas

Aún no podemos visitar las rutas porque los controladores no existen. Sin embargo, sí podemos listarlas para confirmar que Laravel las registra correctamente. En la terminal ejecuta:

```
BASH
php artisan route:list --except-vendor
```

Verás 14 rutas (7 por cada recurso). Cada Route::resource genera exactamente las siguientes rutas:

Verbo HTTP	URI	Acción del controlador	Nombre de ruta
GET	/books	index	books.index
GET	/books/create	create	books.create
POST	/books	store	books.store

Verbo HTTP	URI	Acción del controlador	Nombre de ruta
GET	/books/{book}	show	books.show
GET	/books/{book}/edit	edit	books.edit
PUT/PATCH	/books/{book}	update	books.update
DELETE	/books/{book}	destroy	books.destroy

Cada nombre tiene un propósito específico:

- **index** — listar todos los recursos.
- **create** — mostrar el formulario para crear uno nuevo.
- **store** — recibir el POST del formulario y persistir el recurso.
- **show** — mostrar la ficha de detalle de un recurso identificado por su ID.
- **edit** — mostrar el formulario de edición precargado.
- **update** — recibir el PUT/PATCH y actualizar el recurso.
- **destroy** — recibir el DELETE y eliminar el recurso.

NOTA: Nombres de ruta y la helper route()

El nombre de ruta (books.index, books.show, etc.) permite generar URLs sin acoplarse a las rutas literales:

route('books.show', $book) → http://localhost:8000/books/7

Si más adelante decides cambiar la URI a /catalogo, sólo actualizas la declaración de ruta y todos los enlaces siguen funcionando.

ADVERTENCIA: No hemos creado los controladores aún

Si abres el navegador en /books ahora, verás un error porque BookController no existe. Eso lo solucionamos en la siguiente sección. Lo importante de este paso fue declarar el contrato de rutas que el resto del código respetará.

3.2 BookController: index, show y create

Vamos a crear el controlador de libros como un controlador resource. Laravel generará un esqueleto con los siete métodos prerellenados, listos para que implementemos su lógica.

Paso 1 — Generar el controlador

```bash
php artisan make:controller BookController --resource
```

Esto crea app/Http/Controllers/BookController.php con los siete métodos vacíos. En esta guía implementaremos solo los tres métodos de lectura: index, show y create. Los métodos de escritura (store, update, destroy) los abordaremos en la Guía 7, cuando trabajemos validación y formularios completos.

Paso 2 — Implementar el controlador completo

Reemplaza el contenido del archivo generado con el siguiente código:

APP/HTTP/CONTROLLERS/BOOKCONTROLLER.PHP

```php
<?php

namespace App\Http\Controllers;

use App\Models\Book;
use App\Models\Author;
use App\Models\Category;
use Illuminate\Http\Request;

class BookController extends Controller
{
    /**
     * Lista paginada del catálogo de libros.
     */
    public function index()
    {
        $books = Book::with(['category', 'authors'])
            ->latest()
            ->paginate(12);

        return view('books.index', compact('books'));
    }

    /**
     * Muestra el formulario para crear un nuevo libro.
     */
    public function create()
    {
        $categories = Category::orderBy('name')->get();
        $authors    = Author::orderBy('last_name')->get();

        return view('books.create', compact('categories', 'authors'));
    }

    /**
     * Persiste un libro nuevo. (Pendiente — Guía 7).
     */
    public function store(Request $request)
    {
        // TODO: Validación + creación en Guía 7
    }

    /**
     * Muestra la ficha de detalle de un libro.
     */
    public function show(Book $book)
    {
        $book->load(['authors', 'category', 'activeLoans.member.user']);

        return view('books.show', compact('book'));
    }

    /**
     * Muestra el formulario de edición. (Pendiente — Guía 7).
     */
    public function edit(Book $book)
    {
        // TODO: Formulario de edición en Guía 7
    }
```

```
    /**
     * Actualiza un libro. (Pendiente — Guía 7).
     */
    public function update(Request $request, Book $book)
    {
        // TODO: Validación + actualización en Guía 7
    }

    /**
     * Elimina un libro. (Pendiente — Guía 7).
     */
    public function destroy(Book $book)
    {
        // TODO: Borrado lógico en Guía 7
    }
}
```

Paso 3 — Comprender route model binding

> **DEFINICIÓN: Route model binding implícito**
>
> Cuando el parámetro de ruta {book} y la variable type-hinted del método (Book $book) coinciden en nombre, Laravel ejecuta automáticamente Book::findOrFail($id) y nos entrega ya el modelo.
>
> Si el ID no existe en la base de datos, Laravel responde 404 sin que escribamos una sola línea. Esto elimina código repetitivo y previene errores comunes.

En el método show(Book $book), Laravel hace tres cosas por nosotros:

1. Lee el segmento {book} de la URL (por ejemplo, 7 en /books/7).
2. Ejecuta Book::findOrFail(7) y nos pasa el modelo resuelto.
3. Si no existe, devuelve una respuesta HTTP 404 antes de entrar al método.

Paso 4 — Comprender eager loading

> **DEFINICIÓN: Eager loading y el problema N+1**
>
> El problema N+1 ocurre cuando se ejecuta 1 consulta para listar N libros y luego N consultas adicionales para obtener la categoría/autores de cada uno (1 + N).
>
> Eager loading carga los datos relacionados en pocas consultas adicionales constantes, sin importar cuántos libros haya. with() carga relaciones desde el inicio; load() las carga sobre un modelo ya obtenido.

Método	Cuándo se usa	Ejemplo en BookController
with()	Antes de ejecutar la consulta (constructor)	Book::with(['category', 'authors'])->paginate(12)
load()	Después de tener el modelo ya en memoria	$book->load(['authors', 'category'])

> **EJEMPLO: activeLoans.member.user — eager loading anidado**
>
> La cadena 'activeLoans.member.user' carga, en una sola operación, los préstamos activos del libro, el miembro asociado a cada préstamo y el usuario asociado a cada miembro.

> Esto evita 1 + N + N + N consultas en la vista de detalle, donde mostraremos los préstamos activos de un libro junto al nombre real del miembro.

El método index utiliza paginate(12) en lugar de get(): Laravel devolverá una instancia LengthAwarePaginator con 12 libros por página y el método ->links() en la vista renderizará los controles de paginación automáticamente.

3.3 AuthorController: listado y ficha de autor

Procedemos de manera análoga para el recurso de autores, pero con una diferencia interesante: queremos mostrar cuántos libros tiene cada autor sin cargar todos los libros completos. Para eso usaremos withCount.

Paso 1 — Generar el controlador

```bash
BASH
php artisan make:controller AuthorController --resource
```

Paso 2 — Implementar index, show y placeholders

Reemplaza el contenido del archivo generado con:

```php
APP/HTTP/CONTROLLERS/AUTHORCONTROLLER.PHP
<?php

namespace App\Http\Controllers;

use App\Models\Author;
use Illuminate\Http\Request;

class AuthorController extends Controller
{
    /**
     * Lista paginada de autores con conteo de libros.
     */
    public function index()
    {
        $authors = Author::withCount('books')
            ->orderBy('last_name')
            ->paginate(15);

        return view('authors.index', compact('authors'));
    }

    /**
     * Muestra la ficha del autor con sus libros.
     */
    public function show(Author $author)
    {
        $author->load('books.category');

        return view('authors.show', compact('author'));
    }

    // Métodos placeholder — se implementan en guías posteriores
    public function create() { /* Guía 7 */ }
```

```
    public function store(Request $request) { /* Guía 7 */ }
    public function edit(Author $author) { /* Guía 7 */ }
    public function update(Request $request, Author $author) { /* Guía 7 */ }
    public function destroy(Author $author) { /* Guía 7 */ }
}
```

DEFINICIÓN: withCount y la propiedad books_count

withCount('books') agrega una sub-consulta SQL que cuenta los libros relacionados sin traerlos a memoria. El resultado se expone como una propiedad dinámica con el sufijo _count.

Si la relación se llama books, la propiedad será $author->books_count.

Esto es ideal para listados grandes donde solo necesitamos el número, no los registros completos.

En el método show usamos $author->load('books.category'): cargamos los libros del autor junto con la categoría de cada libro en una sola operación adicional. La vista de detalle podrá iterar sobre $author->books y mostrar el badge de categoría sin disparar consultas adicionales.

3.4 Layout principal: el esqueleto de toda la aplicación

Antes de crear las vistas concretas, definimos el layout que todas heredarán. Un layout en Blade es una vista padre que declara la estructura HTML común (head, navbar, footer) y deja huecos llamados secciones que las vistas hijas rellenan.

> **DEFINICIÓN: Directivas Blade fundamentales**
> @yield('nombre') — declara un hueco con un nombre (ej. @yield('content')).
> @section('nombre') — abre una sección que rellenará un hueco del layout padre.
> @endsection — cierra una sección.
> @extends('vista') — indica que la vista hereda de otro layout.
> @if(...) / @endif — condicionales.
> @foreach(...) / @endforeach — bucles.

Paso 1 — Crear el directorio y el archivo de layout

Crea el directorio resources/views/layouts/ y dentro un archivo app.blade.php. Desde la terminal:

```bash
mkdir -p resources/views/layouts
touch resources/views/layouts/app.blade.php
```

Paso 2 — Escribir el layout completo

Pega el siguiente contenido en resources/views/layouts/app.blade.php:

```
RESOURCES/VIEWS/LAYOUTS/APP.BLADE.PHP
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@yield('title', 'Inicio') - Biblioteca</title>

    {{-- Tailwind CSS vía CDN --}}
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-slate-50 text-slate-800 min-h-screen flex flex-col">

    {{-- ============ NAVBAR ============ --}}
    <nav class="bg-slate-900 text-white shadow-md">
        <div class="container mx-auto px-4 py-3 flex items-center justify-
between">
            <a href="{{ route('books.index') }}" class="text-xl font-bold flex
items-center gap-2">
                <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6"
fill="none"
                     viewBox="0 0 24 24" stroke="currentColor" stroke-
width="2">
                    <path stroke-linecap="round" stroke-linejoin="round"
                          d="M12 6.253v13m0-13C10.832 5.477 9.246 5 7.5 5
                             S4.168 5.477 3 6.253v13C4.168 18.477 5.754 18 7.5
18
                             s3.332.477 4.5 1.253m0-13C13.168 5.477 14.754 5
16.5 5
```

```
                            c1.747 0 3.332.477 4.5 1.253v13C19.832 18.477
18.247 18 16.5 18
                            c-1.746 0-3.332.477-4.5 1.253" />
                </svg>
                Biblioteca
            </a>

            <div class="hidden md:flex items-center gap-6">
                <a href="{{ route('books.index') }}"
                   class="hover:text-amber-300 transition">Catálogo</a>
                <a href="{{ route('authors.index') }}"
                   class="hover:text-amber-300 transition">Autores</a>
            </div>

            {{-- Auth placeholder — se completará en Guía 8 --}}
            <div class="flex items-center gap-3">
                <a href="#" class="text-sm text-slate-300 hover:text-
white">Login</a>
                <a href="#"
                   class="text-sm bg-amber-600 hover:bg-amber-700 px-3 py-1
rounded">
                    Registro
                </a>
            </div>
        </div>
    </nav>

    {{-- ============ MENSAJES FLASH ============ --}}
    @if(session('success'))
        <div class="container mx-auto px-4 mt-4">
            <div class="bg-green-100 border-l-4 border-green-500 text-green-800
                        px-4 py-3 rounded">
                {{ session('success') }}
            </div>
        </div>
    @endif

    @if(session('error'))
        <div class="container mx-auto px-4 mt-4">
            <div class="bg-red-100 border-l-4 border-red-500 text-red-800
                        px-4 py-3 rounded">
                {{ session('error') }}
            </div>
        </div>
    @endif

    {{-- ============ CONTENIDO ============ --}}
    <main class="container mx-auto px-4 py-8 flex-grow">
        @yield('content')
    </main>

    {{-- ============ FOOTER ============ --}}
    <footer class="bg-slate-900 text-slate-300 mt-12 py-6">
        <div class="container mx-auto px-4 text-center text-sm">
            &copy; {{ date('Y') }} Universidad Autónoma Tomás Frías —
            Ingeniería Informática — INF560
        </div>
    </footer>

</body>
</html>
```

> **NOTA: Tailwind CSS vía CDN**
>
> Usar el CDN es perfecto para desarrollo y aprendizaje: no requiere instalación de Node, ni compilación de assets, ni configuración de Vite.
>
> En producción real, sin embargo, se recomienda integrar Tailwind con Vite (preconfigurado en Laravel 13) para purgar clases no usadas y reducir el tamaño del CSS.

3.5 Vistas de libros: index, show y create

Crea el directorio resources/views/books/ y dentro tres archivos:

```bash
mkdir -p resources/views/books
touch resources/views/books/index.blade.php
touch resources/views/books/show.blade.php
touch resources/views/books/create.blade.php
```

Paso 1 — Vista resources/views/books/index.blade.php

Esta vista renderiza el catálogo en una grilla responsiva. Pega el siguiente contenido:

```
RESOURCES/VIEWS/BOOKS/INDEX.BLADE.PHP
@extends('layouts.app')

@section('title', 'Catálogo')

@section('content')
    <div class="flex items-center justify-between mb-6">
        <h1 class="text-3xl font-bold text-slate-900">Catálogo de Libros</h1>
        <a href="{{ route('books.create') }}"
           class="bg-slate-900 hover:bg-slate-800 text-white px-4 py-2 rounded
                  text-sm font-medium">
            + Registrar Libro
        </a>
    </div>

    @if($books->isEmpty())
        <p class="text-slate-500 italic">Aún no hay libros registrados.</p>
    @else
        <div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-
cols-4 gap-6">
            @foreach($books as $book)
                <x-book-card :book="$book" />
            @endforeach
        </div>

        <div class="mt-8">
            {{ $books->links() }}
        </div>
    @endif
@endsection
```

> **NOTA: La etiqueta <x-book-card>**
>
> <x-book-card :book="$book" /> referencia un componente Blade anónimo que crearemos en la sección 3.7.
>
> El prefijo x- es la convención de Blade para componentes; :book con dos puntos pasa la variable PHP $book como prop al componente.

Paso 2 — Vista resources/views/books/show.blade.php

```
RESOURCES/VIEWS/BOOKS/SHOW.BLADE.PHP
@extends('layouts.app')

@section('title', $book->title)
```

```
@section('content')
    <a href="{{ route('books.index') }}"
       class="text-sm text-slate-600 hover:text-slate-900 mb-4 inline-block">
        ← Volver al catálogo
    </a>

    <div class="grid grid-cols-1 lg:grid-cols-3 gap-8">

        {{-- ===== Columna izquierda: portada ===== --}}
        <div class="lg:col-span-1">
            @if($book->cover_url)
                <img src="{{ $book->cover_url }}" alt="{{ $book->title }}"
                     class="w-full rounded shadow-md">
            @else
                <div class="w-full aspect-[2/3] bg-slate-200 rounded flex
items-center
                           justify-center text-slate-400">
                    <svg xmlns="http://www.w3.org/2000/svg" class="h-20 w-20"
fill="none"
                         viewBox="0 0 24 24" stroke="currentColor" stroke-
width="1.5">
                        <path stroke-linecap="round" stroke-linejoin="round"
                              d="M12 6.253v13m0-13C10.832 5.477 9.246 5 7.5 5
                                 S4.168 5.477 3 6.253v13C4.168 18.477 5.754 18
7.5 18" />
                    </svg>
                </div>
            @endif
        </div>

        {{-- ===== Columna derecha: información ===== --}}
        <div class="lg:col-span-2 space-y-4">

            <div>
                <h1 class="text-3xl font-bold text-slate-900">{{ $book->title
}}</h1>
                <p class="text-slate-600 mt-1">
                    @foreach($book->authors as $author)
                        <a href="{{ route('authors.show', $author) }}"
                           class="hover:underline">{{ $author->full_name
}}</a>@if(!$loop->last), @endif
                    @endforeach
                </p>
            </div>

            <div class="flex gap-2 items-center">
                <x-category-badge :category="$book->category" />
                <x-book-status-badge :status="$book->status" />
            </div>

            <dl class="grid grid-cols-2 gap-x-6 gap-y-2 text-sm">
                <dt class="text-slate-500">ISBN</dt>
                <dd>{{ $book->isbn ?? '—' }}</dd>

                <dt class="text-slate-500">Editorial</dt>
                <dd>{{ $book->publisher ?? '—' }}</dd>

                <dt class="text-slate-500">Año de publicación</dt>
                <dd>{{ $book->publication_year ?? '—' }}</dd>
```

```
                <dt class="text-slate-500">Páginas</dt>
                <dd>{{ $book->pages ?? '—' }}</dd>

                <dt class="text-slate-500">Idioma</dt>
                <dd>{{ $book->language ?? '—' }}</dd>
            </dl>

            @if($book->description)
                <div class="pt-2">
                    <h2 class="font-semibold text-slate-900 mb-
1">Descripción</h2>
                    <p class="text-slate-700 leading-relaxed">{{ $book-
>description }}</p>
                </div>
            @endif

            {{-- ===== Disponibilidad ===== --}}
            <div class="bg-slate-100 rounded p-4">
                <h2 class="font-semibold text-slate-900 mb-
1">Disponibilidad</h2>
                <p class="text-slate-700">
                    <span class="text-2xl font-bold text-slate-900">
                        {{ $book->available_copies }}
                    </span>
                    de {{ $book->total_copies }} copias disponibles
                </p>
            </div>

            {{-- ===== Préstamos activos ===== --}}
            @if($book->activeLoans->isNotEmpty())
                <div class="pt-2">
                    <h2 class="font-semibold text-slate-900 mb-2">Préstamos
activos</h2>
                    <table class="w-full text-sm">
                        <thead class="bg-slate-200 text-slate-700">
                            <tr>
                                <th class="text-left px-3 py-2">Miembro</th>
                                <th class="text-left px-3 py-2">Fecha
préstamo</th>
                                <th class="text-left px-3 py-2">Devolución</th>
                            </tr>
                        </thead>
                        <tbody>
                            @foreach($book->activeLoans as $loan)
                                <tr class="border-b border-slate-200">
                                    <td class="px-3 py-2">
                                        {{ $loan->member->user->name ?? '—' }}
                                    </td>
                                    <td class="px-3 py-2">
                                        {{ $loan->loaned_at?->format('d/m/Y')
}}
                                    </td>
                                    <td class="px-3 py-2">
                                        {{ $loan->due_at?->format('d/m/Y') }}
                                    </td>
                                </tr>
                            @endforeach
                        </tbody>
                    </table>
                </div>
            @endif

        </div>
```

```
    </div>
@endsection
```

Paso 3 — Vista resources/views/books/create.blade.php (placeholder)

Esta vista mostrará el formulario completo en la Guía 7. Por ahora maquetamos la estructura visual con campos deshabilitados, para que ya podamos navegar a /books/create y ver una pantalla coherente.

```
RESOURCES/VIEWS/BOOKS/CREATE.BLADE.PHP
@extends('layouts.app')

@section('title', 'Registrar libro')

@section('content')
    <div class="max-w-3xl mx-auto">
        <h1 class="text-2xl font-bold text-slate-900 mb-2">Registrar Libro</h1>
        <p class="text-slate-500 mb-6 italic">
            Formulario de previsualización — la lógica de guardado se
implementa en la Guía 7.
        </p>

        <form class="bg-white rounded shadow-sm p-6 space-y-4">

            <div>
                <label class="block text-sm font-medium text-slate-700 mb-
1">Título</label>
                <input type="text" disabled placeholder="Ej. Cien años de
soledad"
                       class="w-full border border-slate-300 rounded px-3 py-2
bg-slate-100
                              text-slate-500">
            </div>

            <div class="grid grid-cols-1 md:grid-cols-2 gap-4">
                <div>
                    <label class="block text-sm font-medium text-slate-700 mb-
1">
                        ISBN
                    </label>
                    <input type="text" disabled
                           class="w-full border border-slate-300 rounded px-3
py-2 bg-slate-100">
                </div>
                <div>
                    <label class="block text-sm font-medium text-slate-700 mb-
1">
                        Año de publicación
                    </label>
                    <input type="number" disabled
                           class="w-full border border-slate-300 rounded px-3
py-2 bg-slate-100">
                </div>
            </div>

            <div>
                <label class="block text-sm font-medium text-slate-700 mb-
1">Categoría</label>
                <select disabled
```

```
                        class="w-full border border-slate-300 rounded px-3 py-2
bg-slate-100">
                    <option>Selecciona una categoría...</option>
                </select>
            </div>

            <div>
                <label class="block text-sm font-medium text-slate-700 mb-
1">Autores</label>
                <select disabled multiple
                        class="w-full border border-slate-300 rounded px-3 py-2
bg-slate-100 h-32">
                    <option>Selecciona uno o más autores...</option>
                </select>
            </div>

            <div>
                <label class="block text-sm font-medium text-slate-700 mb-
1">Descripción</label>
                <textarea disabled rows="4"
                          class="w-full border border-slate-300 rounded px-3
py-2 bg-slate-100">
                </textarea>
            </div>

            <button type="button" disabled
                    class="bg-slate-400 text-white px-4 py-2 rounded cursor-
not-allowed">
                Guardar (deshabilitado)
            </button>
        </form>
    </div>
@endsection
```

3.6 Vistas de autores: index y show

```bash
mkdir -p resources/views/authors
touch resources/views/authors/index.blade.php
touch resources/views/authors/show.blade.php
```

Paso 1 — Vista resources/views/authors/index.blade.php

Para autores usaremos una tabla en lugar de cards, ya que la información es más tabular:

```
RESOURCES/VIEWS/AUTHORS/INDEX.BLADE.PHP
@extends('layouts.app')

@section('title', 'Autores')

@section('content')
    <h1 class="text-3xl font-bold text-slate-900 mb-6">Autores</h1>

    @if($authors->isEmpty())
        <p class="text-slate-500 italic">Aún no hay autores registrados.</p>
    @else
        <div class="bg-white shadow-sm rounded overflow-hidden">
            <table class="w-full">
                <thead class="bg-slate-900 text-white text-left text-sm">
                    <tr>
                        <th class="px-4 py-3">Nombre completo</th>
                        <th class="px-4 py-3">Nacionalidad</th>
                        <th class="px-4 py-3 text-center">N° de libros</th>
                        <th class="px-4 py-3 text-right">Acciones</th>
                    </tr>
                </thead>
                <tbody>
                    @foreach($authors as $author)
                        <tr class="border-b border-slate-100
                                    {{ $loop->even ? 'bg-slate-50' : 'bg-white'
}}">
                            <td class="px-4 py-3 font-medium">{{ $author-
>full_name }}</td>
                            <td class="px-4 py-3 text-slate-600">
                                {{ $author->nationality ?? '—' }}
                            </td>
                            <td class="px-4 py-3 text-center">
                                <span class="bg-slate-200 text-slate-800 px-2
py-1 rounded
                                            text-xs font-bold">
                                    {{ $author->books_count }}
                                </span>
                            </td>
                            <td class="px-4 py-3 text-right">
                                <a href="{{ route('authors.show', $author) }}"
                                   class="text-sm text-amber-700
hover:underline">Ver →</a>
                            </td>
                        </tr>
                    @endforeach
                </tbody>
            </table>
        </div>

        <div class="mt-6">{{ $authors->links() }}</div>
```

```
    @endif
@endsection
```

Paso 2 — Vista resources/views/authors/show.blade.php

```
RESOURCES/VIEWS/AUTHORS/SHOW.BLADE.PHP
@extends('layouts.app')

@section('title', $author->full_name)

@section('content')
    <a href="{{ route('authors.index') }}"
       class="text-sm text-slate-600 hover:text-slate-900 mb-4 inline-block">
        ← Volver a autores
    </a>

    <div class="bg-white shadow-sm rounded p-6 mb-8">
        <h1 class="text-3xl font-bold text-slate-900">{{ $author->full_name
}}</h1>
        <p class="text-slate-600 mt-1">
            {{ $author->nationality ?? 'Nacionalidad desconocida' }}
            @if($author->birth_date)
                · Nacido el {{ $author->birth_date->format('d/m/Y') }}
            @endif
        </p>

        @if($author->biography)
            <p class="mt-4 text-slate-700 leading-relaxed">{{ $author-
>biography }}</p>
        @endif
    </div>

    <h2 class="text-2xl font-bold text-slate-900 mb-4">
        Libros del autor ({{ $author->books->count() }})
    </h2>

    @if($author->books->isEmpty())
        <p class="text-slate-500 italic">Este autor aún no tiene libros
registrados.</p>
    @else
        <div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-
cols-4 gap-6">
            @foreach($author->books as $book)
                <x-book-card :book="$book" />
            @endforeach
        </div>
    @endif
@endsection
```

NOTA: Reutilización de la card de libros

Observa que tanto books/index.blade.php como authors/show.blade.php usan exactamente el mismo componente <x-book-card :book="$book" />.

Esa es la magia de los componentes: una sola definición se reutiliza en cualquier vista, garantizando consistencia visual y reduciendo duplicación.

3.7 Componentes Blade reutilizables

Los componentes Blade encapsulan trozos de UI con su propia lógica de presentación. Hay dos variantes:

Tipo	Cuándo se usa	Ubicación
Anónimos	Solo plantilla, sin lógica PHP propia	resources/views/components/
De clase	Necesitan lógica, métodos o computed props	app/View/Components/ + view

Para esta guía usaremos componentes anónimos: son archivos Blade puros dentro de resources/views/components/, y se referencian con la sintaxis <x-nombre>. No requieren generar clases PHP.

Paso 1 — Crear el directorio de componentes

```bash
mkdir -p resources/views/components
```

Paso 2 — Componente book-status-badge.blade.php

Crea resources/views/components/book-status-badge.blade.php con el siguiente contenido:

```
RESOURCES/VIEWS/COMPONENTS/BOOK-STATUS-BADGE.BLADE.PHP
@props(['status'])

@php
    $config = [
        'available'   => ['label' => 'Disponible',    'class' => 'bg-green-100
text-green-800'],
        'reserved'    => ['label' => 'Reservado',     'class' => 'bg-yellow-100
text-yellow-800'],
        'unavailable' => ['label' => 'No disponible', 'class' => 'bg-red-100
text-red-800'],
    ];
    $cfg = $config[$status] ?? ['label' => ucfirst($status), 'class' => 'bg-
slate-100 text-slate-800'];
@endphp

<span class="inline-block px-2 py-1 rounded text-xs font-semibold {{
$cfg['class'] }}">
    {{ $cfg['label'] }}
</span>
```

DEFINICIÓN: @props en componentes anónimos

@props(['status']) declara los atributos que el componente acepta como variables PHP. Cuando llamamos <x-book-status-badge :status="$book->status" />, dentro del componente la variable $status ya está disponible.

También se pueden declarar valores por defecto con un array asociativo: @props(['size' => 'md', 'color' => 'blue']).

Paso 3 — Componente category-badge.blade.php

Crea resources/views/components/category-badge.blade.php:

```
RESOURCES/VIEWS/COMPONENTS/CATEGORY-BADGE.BLADE.PHP
@props(['category'])

@if($category)
    <span class="inline-block px-2 py-1 rounded text-xs font-semibold text-
white"
          style="background-color: {{ $category->color ?? '#475569' }}">
        {{ $category->name }}
    </span>
@endif
```

NOTA: Por qué style inline en lugar de clase Tailwind

El color de cada categoría es un valor dinámico almacenado en la base de datos (ej. #DC2626, #0F766E), no una clase predefinida. Tailwind sólo incluye clases que detecta en archivos analizados estáticamente, por lo que bg-[#dc2626] dinámico no funcionaría.

El atributo style="background-color: {{ ... }}" es la forma idiomática de aplicar valores realmente dinámicos.

Paso 4 — Componente book-card.blade.php

Este es el componente más importante: la card que se reutiliza en el catálogo y en la ficha de autor.

```
RESOURCES/VIEWS/COMPONENTS/BOOK-CARD.BLADE.PHP
@props(['book'])

<a href="{{ route('books.show', $book) }}"
   class="block bg-white rounded shadow-sm hover:shadow-md transition overflow-
hidden">

    {{-- Portada --}}
    @if($book->cover_url)
        <img src="{{ $book->cover_url }}" alt="{{ $book->title }}"
             class="w-full aspect-[2/3] object-cover">
    @else
        <div class="w-full aspect-[2/3] bg-slate-200 flex items-center justify-
center
                    text-slate-400">
            <svg xmlns="http://www.w3.org/2000/svg" class="h-12 w-12"
fill="none"
                 viewBox="0 0 24 24" stroke="currentColor" stroke-width="1.5">
                <path stroke-linecap="round" stroke-linejoin="round"
                      d="M12 6.253v13m0-13C10.832 5.477 9.246 5 7.5 5
                         S4.168 5.477 3 6.253v13C4.168 18.477 5.754 18 7.5 18"
/>
            </svg>
        </div>
    @endif

    {{-- Información --}}
    <div class="p-3 space-y-2">
        <h3 class="font-semibold text-slate-900 line-clamp-2" title="{{ $book-
>title }}">
            {{ $book->title }}
        </h3>

        <p class="text-xs text-slate-600 line-clamp-1">
            @foreach($book->authors as $author)
```

```
                {{ $author->full_name }}@if(!$loop->last), @endif
            @endforeach
        </p>

        <div class="flex flex-wrap gap-1">
            <x-category-badge :category="$book->category" />
            <x-book-status-badge :status="$book->status" />
        </div>

        <p class="text-xs text-slate-500">
            {{ $book->publication_year ?? 's/f' }}
        </p>
    </div>
</a>
```

EJEMPLO: Componentes que usan otros componentes

Dentro de book-card.blade.php usamos <x-category-badge> y <x-book-status-badge>.

Los componentes Blade son perfectamente componibles: un componente puede invocar a cualquier otro componente, igual que los componentes en React o Vue.

Paso 5 — Probar todo en el navegador

Levanta el servidor de desarrollo y verifica cada ruta:

```bash
php artisan serve
```

Visita en el navegador:

- http://localhost:8000/ → debe redirigir a /books
- http://localhost:8000/books → grilla de cards con paginación
- http://localhost:8000/books/1 → ficha de detalle del primer libro
- http://localhost:8000/books/create → formulario placeholder
- http://localhost:8000/authors → tabla de autores con conteo de libros
- http://localhost:8000/authors/1 → ficha de autor con sus libros

ADVERTENCIA: Si una vista falla

Revisa los nombres de las relaciones en tus modelos: la vista de detalle usa $book->activeLoans y la del autor usa $author->books. Si en tu Guía 5 nombraste estas relaciones de manera diferente (por ejemplo, loans en lugar de activeLoans), ajusta los controladores y vistas en consecuencia, o renombra las relaciones del modelo.

Verifica que los seeders hayan poblado la base: php artisan migrate:fresh --seed.

3.8 Versionado: commit y tag v0.3.0

Cuando todas las rutas funcionan correctamente y has revisado las vistas en el navegador, registra el avance en Git:

```bash
# 1. Verifica los archivos modificados/creados
git status

# 2. Agrega todos los cambios
git add .

# 3. Commit semántico
git commit -m "feat: rutas resource, controladores y vistas Blade con
componentes"

# 4. Tag anotado de la entrega
git tag -a v0.3.0 -m "Guía 6: Rutas, controladores y vistas Blade"

# 5. Push al repositorio remoto incluyendo tags
git push origin main --tags
```

DEFINICIÓN: Commit semántico

Convención que prefija el mensaje con un tipo: feat: (nueva funcionalidad), fix: (corrección), refactor: (reorganización sin cambio de comportamiento), docs:, test:, chore:.

Hace que el historial sea legible y permite generar changelogs automáticos.

DEFINICIÓN: Tag anotado vs ligero

git tag -a v0.3.0 -m "..." → tag anotado: incluye autor, fecha y mensaje, se almacena como objeto completo.

git tag v0.3.0 → tag ligero: solo apunta a un commit.

Para entregas académicas siempre usa tags anotados con -a.

4. Ejercicios propuestos

Estos ejercicios consolidan lo aprendido. Resuélvelos antes de la fecha de entrega y déjalos en commits adicionales después del tag v0.3.0.

Ejercicio 1 — CategoriesController y vistas de categoría

Implementa el controlador y vistas para el recurso de categorías:

1. Genera CategoriesController con php artisan make:controller CategoriesController --resource.
2. Implementa index() devolviendo Category::withCount('books')->orderBy('name')->paginate(20).
3. Implementa show(Category $category) cargando $category->load('books.authors').
4. Registra Route::resource('categories', CategoriesController::class) en routes/web.php.
5. Crea resources/views/categories/index.blade.php con una grilla de cards donde cada card muestre el nombre con el color de la categoría como fondo y el número de libros en ella.

6. Crea resources/views/categories/show.blade.php que muestre los libros de esa categoría reutilizando el componente <x-book-card>.
7. Agrega un enlace 'Categorías' al navbar del layout principal.

Ejercicio 2 — Componente membership-badge

Crea resources/views/components/membership-badge.blade.php que reciba la prop type y muestre:

- standard → 'Estándar' con fondo gris (bg-slate-200 text-slate-800).
- premium → 'Premium' con fondo dorado (bg-amber-200 text-amber-900) y un pequeño icono ★.
- student → 'Estudiante' con fondo azul (bg-blue-100 text-blue-800).

Pruébalo creando una vista temporal /miembros (ruta cerrada y vista simple) que liste los primeros 10 miembros con su nombre y el badge correspondiente. No es necesario crear el MembersController completo aún; basta con una closure en routes/web.php para verificar que el componente funciona.

> **NOTA: Sugerencia de uso**
> Una closure de prueba en routes/web.php podría ser:
> Route::get('/miembros', fn() => view('miembros.index', ['members' => App\Models\Member::with('user')->take(10)->get()]));
> Recuerda eliminarla o moverla a un controlador real más adelante.

5. Rúbrica de evaluación

La calificación se asigna sobre 100 puntos según los siguientes criterios objetivos:

Criterio	Descripción	Puntos
Rutas resource	books y authors registradas con Route::resource y ruta raíz redirigiendo a /books	10
BookController	Métodos index, show y create implementados con eager loading correcto	20
AuthorController	Métodos index (con withCount) y show implementados correctamente	10
Layout principal	app.blade.php con navbar funcional, mensajes flash y footer	10
Vista books/index	Grid responsive de cards y paginación funcionando	15
Vista books/show	Detalle completo: portada, info, disponibilidad y préstamos activos	10
Vistas authors/*	index con tabla y show con datos del autor más sus libros	5
Componentes Blade	book-status-badge, category-badge y book-card creados y usados	10
Ejercicios propuestos	Ambos ejercicios entregados y funcionales	5

Criterio	Descripción	Puntos
Git tag v3.0	Commit semántico y tag anotado pusheado al remoto	5

TOTAL: 100 puntos

6. Entregables y tag de Git

El estudiante debe entregar:

- URL del repositorio Git público (GitHub, GitLab o Bitbucket) con el tag v3.0 visible.
- El repositorio debe contener los archivos: routes/web.php, app/Http/Controllers/BookController.php, app/Http/Controllers/AuthorController.php, resources/views/layouts/app.blade.php, las vistas books/{index,show,create}.blade.php, las vistas authors/{index,show}.blade.php y los tres componentes en resources/views/components/.
- Para los ejercicios propuestos: CategoriesController, vistas categories/{index,show}.blade.php, componente membership-badge.blade.php y, opcionalmente, una vista temporal de prueba para los miembros.
- Una captura de pantalla del navegador mostrando la vista /books con la grilla de cards funcionando.

ADVERTENCIA: Verificación final antes de entregar

1. php artisan route:list --except-vendor muestra al menos 14 rutas (books + authors).
2. http://localhost:8000/ redirige a /books sin error.
3. La grilla de cards aparece con datos reales (no placeholders) y la paginación es navegable.
4. Al hacer clic en una card, la ficha de detalle muestra el libro con sus autores como enlaces clicables.
5. git push origin main --tags termina sin errores y v0.3.0 es visible en el repositorio remoto.